I A M - L O C A L

Cahier des charges technique et fonctionnel

Plateforme IAM « IAM ACSI »

Gestion des identités et des accès, SSO & MFA

Version	1.0
Date	28 septembre 2026
Statut	Validé — implémentation livrée (PHP natif + MySQL)
Documentation incluse	Diagramme de classes · Diagrammes d'activité (SSO, Connexion & MFA)

Sommaire

1. Contexte et objectifs
2. Périmètre fonctionnel
3. Diagramme de classes
4. Diagrammes d'activité
5. Architecture technique et sécurité
6. Environnement et exigences
7. Livrables et jalons

1. Contexte et objectifs

La structure met à disposition plusieurs applications Web internes (gestion de présence, gestion de salaire, gestion scolaire, etc.), chacune gérée historiquement par son propre mécanisme de connexion et d'utilisateurs. Ce cloisonnement impose une multiplicité de comptes, de mots de passe et de tables d'utilisateurs à maintenir.

Le projet **IAM-Local (IAM ACSI)** vise à centraliser :

- **l'Identité** — un référentiel unique d'utilisateurs, avec organisation hiérarchique et statut de compte ;
- **la Politique de sécurité** — politique de mot de passe, verrouillage anti force brute, expiration, sessions ;
- **la Vérification renforcée (MFA)** — code OTP à usage unique envoyé par e-mail (SMTP), affiché en mode démonstration ;
- **l'Accès (SSO)** — Single Sign-On par jeton à usage unique : une connexion IAM donne accès aux applications enregistrées (accès fluide via le portail, ressaisie facultative sur lien direct) ;
- **la Gouvernance (RBAC)** — rôles, permissions, affectations utilisateur / rôle / application, journal d'audit complet ;
- **l'Administration graphique** — une console de gestion ne nécessitant aucune manipulation SQL en exploitation.

Périmètre. Le présent document décrit l'existant implémenté (version 1.0.0), qui sert de socle au déploiement et aux évolutions ultérieures.

2. Périmètre fonctionnel

2.1 Authentification

- Connexion par identifiant (e-mail) et mot de passe haché (bcrypt via `password_hash`).
- Comptes avec statuts `active` / `inactive` / `locked` / `pending`, premier mot de passe imposé (`must_change_password = 1`), expiration de mot de passe configurable.
- Anti force brute : 5 échecs autorisés, puis verrouillage de 30 min (tentatives journalisées dans `login_attempts`).
- Double vérification MFA par e-mail : OTP de 6 chiffres, valable 5 minutes, 5 essais maximum (`MFA_FORCE_ALL = true`, sauf administrateurs).
- Session PHP dédiée `iam_session` (durée 3600 s, inactivité 1800 s), sessions révocables depuis l'administration.

2.2 SSO applicatif

- Point d'entrée `sso/authorize.php?app=CODE&return=URL[&prompt=login]` : vérifie l'application, la session IAM, le MFA, l'expiration du mot de passe et l'accès de l'utilisateur, puis émet un jeton à usage unique (60 s, enregistré dans `sso_tokens`).
- Fichier garde généré `iam-sso-<code>.php` (déployé automatiquement par l'administrateur ou téléchargeable ; chargé en `auto_prepend_file` via `.htaccess`) : session isolée côté application, validation du jeton via `api/sso/validate.php`, reconstruction de l'utilisateur (rôles, permissions).

- Paramètre par application `sso_prompt` (administré graphiquement) : ajoute `prompte=login` sur accès direct, ce qui impose la ressaisie des identifiants IAM ; via le portail, l'accès reste fluide.
- Vérification de la légitimité de l'URL de retour (même hôte que l'application) avant émission du jeton.

2.3 Portail et applications

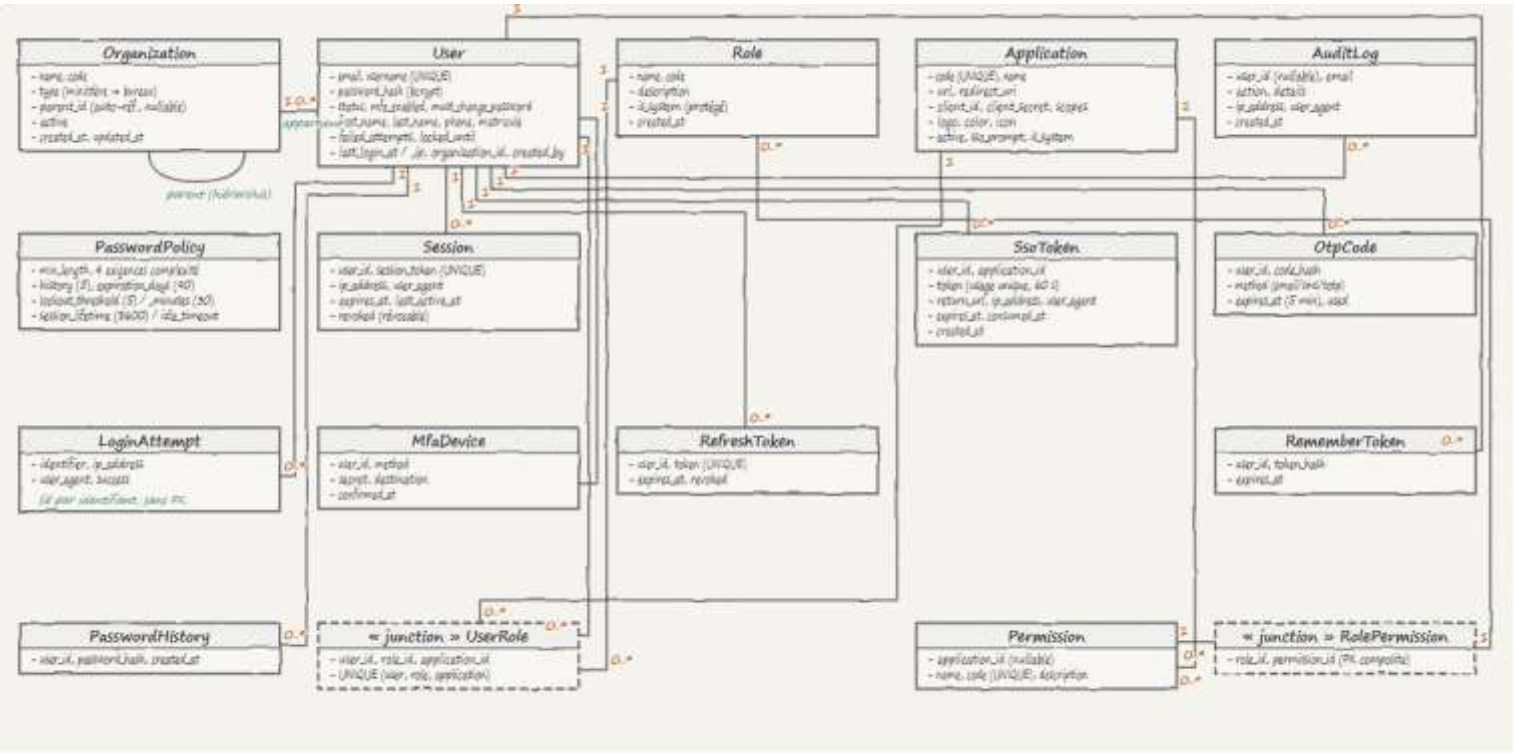
- **Portail** (page d'accueil) : grille d'applications autorisées avec icône, couleur et description ; accès en un clic (lien SSO).
- **Catalogue des applications** : code, nom, URL, `redirect_uri`, `client_id` / `client_secret`, `active`, `sso_prompt`, `is_system`.

2.4 Gouvernance (RBAC) et administration graphique

- **Rôles et permissions** : création, édition et suppression de rôles (rôles système protégés), attribution de permissions par application.
- **Utilisateurs** : création, modification (statut, organisation), réinitialisation de mot de passe, suppression.
- **Organisations** : arborescence Ministère / Direction / Service / Bureau avec affectation des utilisateurs.
- **Sessions** : consultation, filtrage par utilisateur, révocation individuelle ou totale.
- **Journal d'audit** : recherche textuelle et filtrage par action sur `audit_logs`.
- **Paramètres** : politique de mots de passe, durée de session et d'inactivité.
- **E-mail et MFA** : configuration SMTP (Gmail / Outlook, clé applicative), envoi de test, guide pas à pas en 3 étapes.
- **Applications** : création, édition, permissions, accès utilisateurs, déploiement / retrait du garde SSO, téléchargement du garde.

2.5 API et intégration

- **API JSON** : `api/login.php`, `api/sso/validate.php`, `api/sso/app_config.php`, `api/userinfo.php`, gestion des utilisateurs, rôles, permissions, applications et sessions, audit, réinitialisation de mot de passe, MFA (envoi / vérification).
- **Bibliothèque client** : `client/iam_client.php` (helper `iam_client_requires_prompt()`, etc.) et `client/sso_handler.php`.



3. Diagramme de classes

Explication détaillée des classes

Ce modèle décrit un système d'authentification centralisée (SSO) avec gestion des rôles et des droits. Il est inspiré d'une administration, avec ses ministères et ses bureaux.

Le cœur : identité et organisation

Organization représente la structure administrative (ministère, direction, bureau).

- type précise le niveau (ministère → bureau).
- parent_id est nullable et pointe vers la même table, ce qui crée une hiérarchie en arbre (c'est la boucle « parent (hiérarchie) » du schéma d'origine).
- Elle a aussi code, name, active, created_at et updated_at.
- Une organisation contient plusieurs utilisateurs (1 → 0..*), d'où le lien « appartient à ».

User est la table centrale.

- Identité : email, username (UNIQUE), first_name, last_name, phone, matricule.
- Mot de passe : password_hash est haché en bcrypt, jamais en clair. must_change_password force le changement à la première connexion.
- État du compte : status, mfa_enabled, failed_attempts et locked_until (verrouillage temporaire).
- Traçabilité : last_login_at, last_login_ip, organization_id, created_by.

Les droits (RBAC)

Role est un rôle nommé (name, code, description). Le champ is_system protège les rôles vitaux, qu'on ne peut donc ni supprimer ni modifier.

Application est une application cliente qui utilise le SSO.

- code est unique et name est son nom.
- url et redirect_uri servent à rediriger l'utilisateur après connexion.
- client_id, client_secret et scopes suivent le fonctionnement d'OAuth.
- logo, color, icon servent à l'affichage.
- active, sso_prompt et is_system règlent l'état et le comportement.

Permission est un droit précis (name, code UNIQUE, description). application_id est nullable : une permission peut appartenir à une application, ou être globale si le champ est vide.

UserRole (table de jonction) dit quel utilisateur a quel rôle dans quelle application, via user_id, role_id et application_id. La contrainte UNIQUE (user, role, application) empêche les doublons. Le même utilisateur peut donc être « admin » dans une application et « lecteur » dans une autre.

RolePermission (table de jonction) dit quelles permissions contient un rôle. Sa clé primaire est composite (role_id, permission_id).

Le chemin complet est donc : User → UserRole → Role → RolePermission → Permission.

Sessions et jetons

Session est une connexion active. Elle contient session_token (UNIQUE), ip_address, user_agent, expires_at, last_active_at et revoked (pour la révoquer). Ses durées viennent de PasswordPolicy (3600 s de durée de vie, plus un délai d'inactivité).

RefreshToken est un jeton unique et durable qui permet d'obtenir un nouvel accès sans se reconnecter. Il a expires_at et revoked.

SsoToken est un jeton à usage unique valable 60 secondes, échangé lors de la redirection d'une application vers le SSO. Il est lié à user_id et application_id, garde return_url, ip_address et user_agent, et consumed_at empêche sa réutilisation. C'est la seule table de sécurité rattachée à la fois à User et à Application.

RememberToken est le cookie « rester connecté ». Seul le hash (token_hash) est stocké, avec une date d'expiration.

Authentification multi-facteurs

MfaDevice est un appareil de second facteur : method, secret, destination (e-mail ou numéro) et confirmed_at (activation validée).

OtpCode est un code à usage unique valable 5 minutes. Il a code_hash, method (email, sms ou totp) et used.

Sécurité et audit

PasswordPolicy contient les règles de mot de passe et de session :

- longueur minimale et 4 exigences de complexité ;
- historique de 5 mots de passe et expiration au bout de 90 jours ;
- verrouillage après 5 échecs, pendant 30 minutes ;
- durée de session (3600 s) et délai d'inactivité.

Elle n'a aucun lien avec les autres tables : le code la lit simplement comme configuration.

PasswordHistory garde les anciens hash (password_hash, created_at) pour interdire la réutilisation des 5 derniers mots de passe.

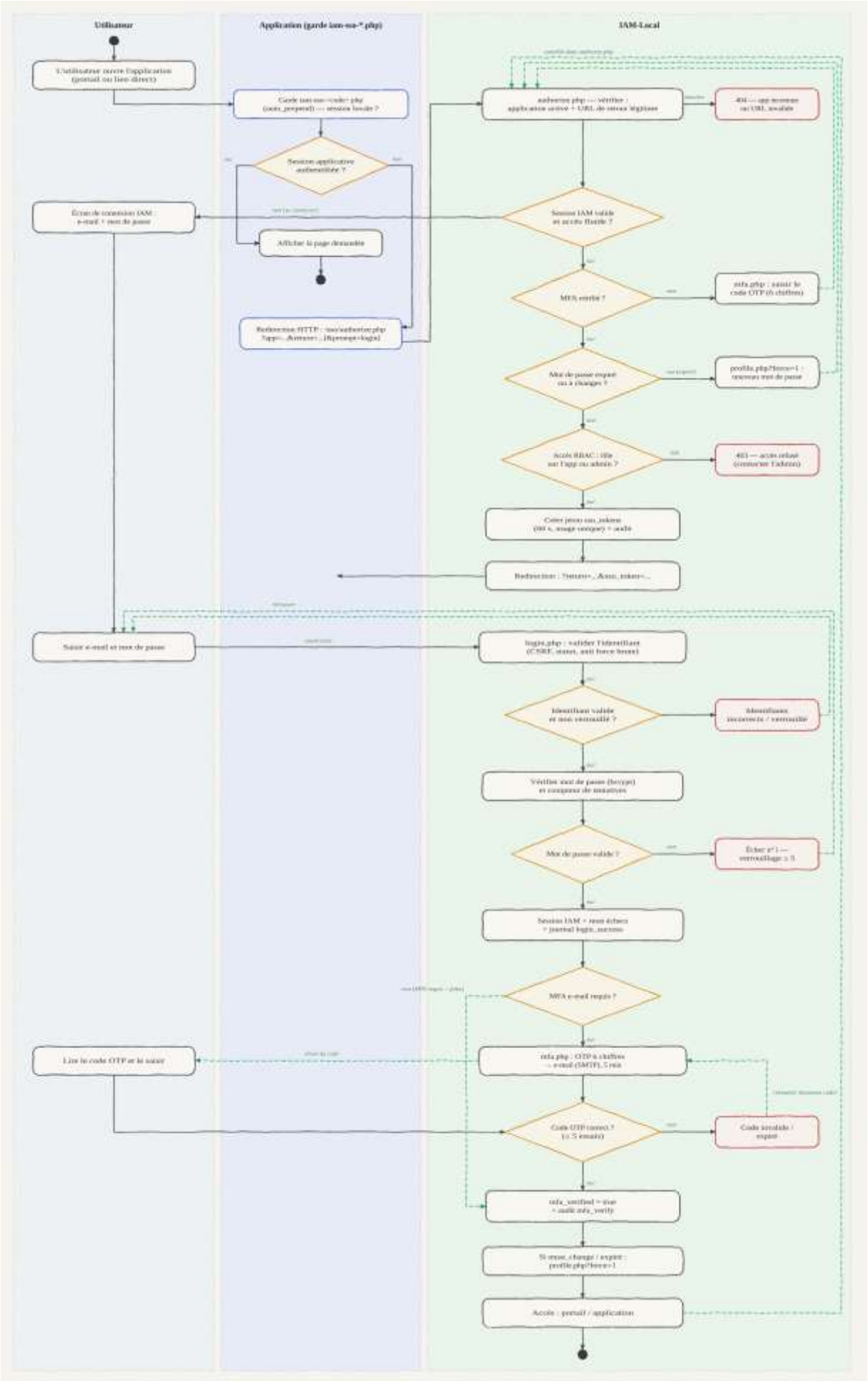
LoginAttempt enregistre chaque tentative (identifier, ip_address, user_agent, success). Elle est « liée par identifiant, sans FK », comme l'indique une note du schéma. On peut ainsi journaliser même un identifiant inexistant, ce qui aide à détecter les attaques.

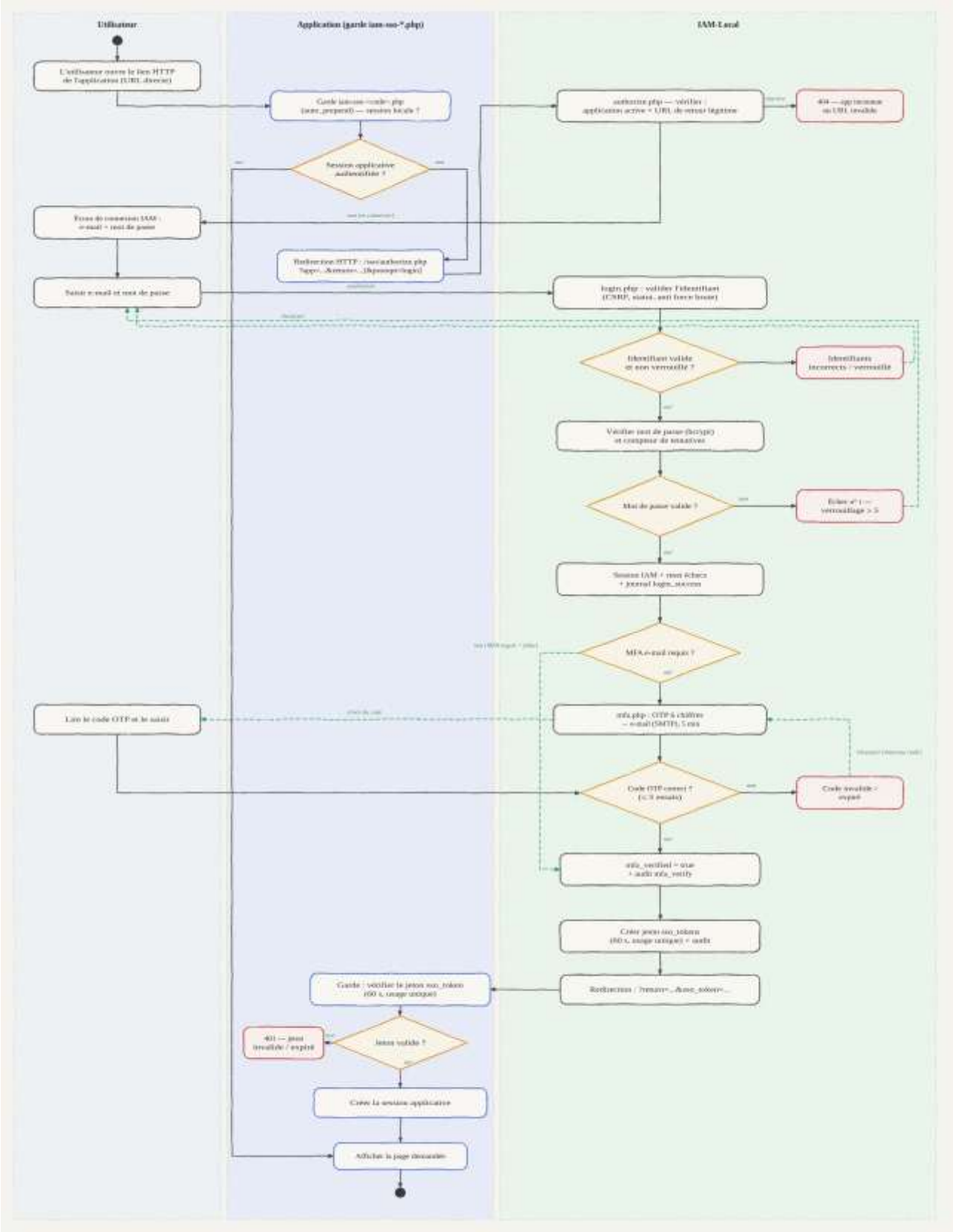
AuditLog est le journal d'audit : user_id (nullable), email, action, details, ip_address, user_agent, created_at. user_id est nullable pour garder la trace d'actions sans utilisateur identifié, ou d'un utilisateur supprimé.

4. Diagrammes d'activité

Deux diagrammes décrivent les flux critiques : **4.1** le flux SSO de bout en bout (portail, application, IAM) et **4.2** la phase d'authentification avec MFA par e-mail et changement de mot de passe.

Lecture : un trait plein représente le flux normal ; un trait en pointillés, un retour ou une nouvelle tentative (re-contrôle dans `authorize.php`) ; un double contour, une sortie en erreur.


Utilisateur
Application (garde iam-sso-* php)
IAM-Local
L'utilisateur ouvre l'application (portail ou lien direct)
Garde iam-sso-<code> php (sans_portail) — session locale ?
Session applicative authentifiée ?
Écran de connexion IAM : e-mail + mot de passe
Afficher la page demandée
Redirection HTTP : /iam/authorize.php ?app=…&return=…(&prompt=login)
authorize.php — vérifier : application active + URL de retour légitime
404 — app inconnue ou URL invalide
Session IAM valide et active (liaison) ?
MFA active ?
mfa.php : saisir le code OTP (6 chiffres)
Mot de passe expiré ou à changer ?
profile.php?force=1 : nouveau mot de passe
Accès RBAC : rôle sur l'app ou admin ?
403 — accès refusé (contourne l'admin)
Créer jeton sso_tokens (60 s, usage unique) + audit
Redirection : ?return=…&sso_token=…
Saisir e-mail et mot de passe
login.php : valider l'identifiant (CSRF, statut, anti force brute)
Identifiant valide et non verrouillé ?
Identifiants incorrects / verrouillé
Vérifier mot de passe (bcrypt) et compteurs de tentatives
Mot de passe valide ?
Échec n° i — verrouillage à 5
Session IAM + reset échecs + journal login_success
MFA e-mail requis ?
Lire le code OTP et le saisir
mfa.php : OTP 6 chiffres — e-mail (SMTP), 5 min
Code OTP correct ? (≤ 5 essais)
Code invalide / expiré
mfa_verified = true + audit mfa_verify
Si must_change / expiré : profile.php?force=1
Accès : portail / application


Utilisateur
Application (garde iam-sso-*.php)
IAM-Local
Session applicative authentifiée ?
Identifiants incorrects / verrouillé
Mot de passe valide ?
Session IAM + reset échecs + journal login_success
MFA e-mail requis ?
Code OTP correct ? (≤ 3 essais)
Code invalide / expiré
mfa_verified = true + audit mfa_verify
Jeton valide ?
Crée la session applicative
Affiche la page demandée

5. Architecture technique et sécurité

5.1 Pile technique

Composant	Choix
Langage	PHP natif (aucun framework), PHP 8.x
Base de données	MySQL 8 (PDO), base `iam_local`, jeu de caractères `utf8mb4`
Sessions	Sessions PHP (cookie `iam_session`), régénération d'ID, HttpOnly, Strict Mode
Mots de passe	bcrypt (`password_hash`), historique anti-réutilisation, expiration configurable
MFA	OTP par e-mail (SMTP : Gmail / Outlook, clé applicative) ; affichage en mode démonstration si SMTP non configuré
Jeton SSO	Jeton aléatoire à usage unique (`sso_tokens`), durée de vie 60 s, consommé après usage

5.2 Mesures de sécurité

- Anti force brute paramétrable (verrouillage après 5 échecs pendant 30 min), journalisé.
- Protection CSRF sur toutes les actions d'administration (jeton de session).
- Échappement HTML systématique (`htmlspecialchars`) sur les saisies restituées.
- Vérification de l'hôte dans l'URL de retour SSO (anti redirection ouverte).
- cURL côté application : vérification SSL activée (`CURLOPT_SSL_VERIFYPEER`).
- Journal d'audit horodaté : connexions, MFA, SSO, administration (création, suppression, droits…).
- Rôles système et applications système non supprimables.

6. Environnementet exigences

Poste	Exigence
Serveur	WAMP (Apache + PHP 8 + MySQL 8), racine `C:\wamp64\www`
Base	`iam_local` — schéma fourni (`sql/schema.sql`), installateur web `install.php`
Applications intégrées (SSO)	Gestion de présence (`GestionPresnceAdenIAM`), gestion de salaire (`GestionSalaireAden`), EduCongo2, etc. — garde déployé par l'administrateur
Comptes de démonstration	Mots de passe documentés dans `docs/guide_iam.html` (admin, RH présence, agent salaire)